ProcureLab

Installation and demo guide

ProcureLab is a multi-market vendor management and RFP platform with three roles: administrator, procurement manager and vendor. It runs in 4 markets (Netherlands, Belgium, Denmark, Germany) and 4 languages (English, Dutch, Danish, German). It stores everything as JSON files and real uploaded files inside the project folder, so you can show exactly where data goes.

It is built as the system under test for an agentic test automation platform. Every screen has stable test selectors, there is a token-protected API for test data, and there are feature flags per market. All companies and people in the demo data are fictional.

Item	Value
Version	1.0.0
Stack	Node.js 20+ (22 LTS recommended), Express, server-rendered HTML, no database
Start command	npm install, then npm start, then open http://localhost:3000
Default login	admin@procurelab.test / Demo123!
Guide date	08 October 2026

Contents

1. Prerequisites
2. Install and run locally
3. Accounts
4. Where data and files are stored
5. Configuration
6. Feature tour and demo script
7. Test hooks and API
8. Deploy online (Render or Docker)
9. Troubleshooting
10. Known limits

1. Prerequisites

Tool	Version	Check with
Node.js	20.6 or newer (22 LTS recommended)	node -v
npm	Comes with Node.js	npm -v
Git	Any recent version (only needed to push to GitHub)	git --version
A browser	Chrome, Edge, Firefox or Safari	
Docker (optional)	Only if you deploy as a container	docker --version

Download Node.js from nodejs.org and pick the LTS installer. On macOS you can also use Homebrew (`brew install node`). On Windows, the installer adds Node.js to your PATH. Open a new terminal after installing.

2. Install and run locally

1. Unzip `procurelab.zip` to a folder, for example `C:\dev\procurelab` or `~/dev/procurelab`.
2. Open a terminal in that folder (in VS Code: Terminal, then New Terminal).
3. Install the dependencies. This takes about 10 seconds:

```
npm install
```

4. Start the app:

```
npm start

# Output on first start:
# [data] first start: demo data written to .../procurelab/data
# ProcureLab v1.0.0 on http://localhost:3000 (default market NL)
```

5. Open http://localhost:3000 and sign in with **admin@procurelab.test** and the password **Demo123!**.

> **First start creates the data.** If the `data` folder has no JSON files, the app writes the demo data and demo PDFs. Every later start loads what is already there, so your changes survive a restart.

Stop the app with Ctrl+C. Use `npm run dev` to restart automatically when you change code.

3. Accounts

All demo accounts use the password **Demo123!**. Passwords are stored hashed (scrypt), never in plain text.

Email	Role	What it shows
admin@procurelab.test	Administrator	Everything a procurement manager sees, plus Users (create, activate and deactivate users) and Data (JSON files, export, reset)
buyer@procurelab.test	Procurement manager	Dashboard, projects, requirements, vendors, RFPs, proposal comparison, AI analyzer
vendor.nl@procurelab.test	Vendor (CloudTech Solutions, NL)	Vendor portal: My RFPs, documents, questions, proposal submission
vendor.be@ / vendor.dk@ / vendor.de@procurelab.test	Vendor (first vendor of each market)	Same portal, in Belgian, Danish or German context
locked@procurelab.test	Procurement manager, deactivated	Login is refused with a 'deactivated' message (negative test)

New users created by the admin are saved to `data/users.json` straight away and can sign in immediately. A vendor user is always linked to one vendor company and is locked to that company's market.

4. Where data and files are stored

Everything lives in the `data` folder inside the project, with one JSON file per collection:

```
procurelab/
  data/
    meta.json          id counters and last-saved time
    users.json         users (password hashes, never plain text)
    vendors.json       vendor companies
    projects.json      projects and assigned vendors
    requirements.json  requirements per project (must / should / could)
    rfps.json          RFPs: requirements in scope, invited vendors, deadline
    proposals.json     proposals: price, VAT, delivery, status
    questions.json     vendor questions and buyer answers
    files.json         metadata of every uploaded file
    activity.json      activity feed shown on the dashboard
    uploads/           the uploaded files themselves (PDF, Word, Excel ...)
```

Action in the app	Where you see it on disk
Admin creates a user	New entry in data/users.json
Procurement creates a project or requirement	data/projects.json, data/requirements.json
Procurement sends an RFP with attachments	data/rfps.json, data/files.json, new files in data/uploads/
Vendor submits a proposal with documents	data/proposals.json, data/files.json, new files in data/uploads/
Anything that happens	data/activity.json and data/meta.json (updatedAt)

How it is saved. Each change is written immediately. Writes are atomic: the app writes a temporary file, then renames it, so a crash never leaves half-written JSON. The admin **Data** page shows every file with its record count, the upload folder size, and the last-saved time.

Uploads. Allowed types are PDF, DOCX, XLSX, PPTX, TXT, CSV, PNG and JPG, up to 5 MB per file and 5 files per form. Other types and larger files are rejected with a clear message. Downloads are permission-checked: a vendor can download documents of RFPs they are invited to and their own proposal files, but never a competitor's proposal.

Backup and reset.

- **Export:** in the app, go to Data, then Download JSON export. Password hashes are left out of the export.
- **Reset from the app:** Data, then Reset demo data. This deletes your changes and uploads and restores the demo data.
- **Reset from the command line:** `npm run reset-data`, or delete the data folder and start again.
- **Different location:** set `DATA_DIR`, for example to a mounted disk in production.

5. Configuration

Every setting has a working default, so configuration is optional. To change one, copy `.env.example` to `.env`, edit it, and start with:

```
npm run start:env      # runs: node --env-file=.env server.js  (Node 20.6+)
```

Alternatively, set the variables in your shell or in your hosting dashboard.

Variable	Default	Purpose
PORT	3000	HTTP port
APP_NAME	ProcureLab	Name shown in the header, login page and page titles
DATA_DIR	./data	Folder for the JSON files and uploads
DEMO_PASSWORD	Demo123!	Password given to the demo accounts when the data is created (see the troubleshooting section)
SESSION_SECRET	dev-only-secret	Signs the session cookie. Always set this online
TEST_API_TOKEN	local-test-token (dev only)	Required in the x-test-token header for the test data API. With NODE_ENV=production and no token, the API is disabled
DEFAULT_MARKET	NL	Market before anyone chooses one
MAX_UPLOAD_MB	5	Upload size limit per file
ALLOW_FLAG_OVERRIDES	true	Allows ?ff_flag=on|off overrides in the URL
FLAG_OVERRIDES	(none)	JSON applied to all markets, for example {"vatBreakdown":false}
ANTHROPIC_API_KEY	(none)	Optional. Enables 'Generate AI summary' in the proposal analyzer
ANTHROPIC_MODEL	claude-haiku-5-5	Model for the AI summary. Check the current model names in the Anthropic docs

Markets and feature flags

Market	Language	Currency	VAT	vatBreakdown	aiAnalyzer	vendorQuestions
NL	Dutch	EUR	21%	on	on	on
BE	Dutch	EUR	21%	off	off	on
DK	Danish	DKK	25%	on	on	off
DE	German	EUR	19%	on	off	off

The language can be chosen independently of the market, so 'DK market in English' shows English text with Danish currency and date formats. Switching market resets the language to that market's default.

6. Feature tour and demo script (about 8 minutes)

This script shows every feature in a logical order. Use two browser windows, or a normal and a private window, so the procurement manager and the vendor are signed in at the same time.

A. Administration (admin@procurelab.test)

1. Open **Users** and create a procurement manager, for example 'Pasha Tester' with home market BE and a password of at least 8 characters.
2. Create a vendor user linked to 'DataSphere'. Note that the market follows the vendor company.
3. Open `data/users.json` in your editor to show both new users, with hashed passwords.
4. Deactivate a user, and show that this user is signed out on their next click.

B. Procurement (buyer@procurelab.test)

1. **Dashboard:** KPI cards (Projects, Active RFPs, Proposals, Vendors) and recent activity.
2. **Projects:** create 'Network Refresh 2027'. Try an end date before the start date and a duplicate name to show validation.
3. Add requirements with priorities (Must, Should, Could), then assign a vendor to the project.
4. **New RFP:** select requirements and vendors, set a deadline, upload a PDF, then Send RFP. Show the new file in `data/uploads/`.
5. **Vendors:** search, filter by expertise and minimum rating, add a vendor, open its details and its RFP history.

C. Vendor portal (vendor.nl@procurelab.test)

1. **My RFPs:** 'Security Assessment RFP' is awaiting response.
2. Open it, download the scope document, and ask a question.
3. Submit a proposal with price, delivery time in months, and an uploaded document. The VAT preview updates while you type.
4. Try submitting again to show the duplicate protection, and open a non-invited RFP URL to show the 404.

D. Comparison and AI Proposal Analyzer (back as procurement)

1. Open 'Cloud Migration RFP' (3 proposals). Sort by price, delivery or rating, and filter by minimum rating.
2. Tick 2 or 3 proposals and click Compare selected for a side-by-side view.
3. **AI Proposal Analyzer:** scores weigh price 50%, delivery 30% and rating 20%. DigitalWorks is flagged 'Missing: proposal document', which raises its risk.
4. Click Generate AI summary. Without an API key, the app shows a clear 'not configured' notice. With a key, it shows an LLM summary in the current language.
5. Select CloudTech Solutions. The other proposals become 'Not selected', and the RFP becomes 'Awarded'. The dashboard activity updates.

E. Multi-market and language

1. Switch market to DK, then the language to English. Amounts show in DKK with 25% VAT.
2. Switch to DE and open the German RFP form. The 'Send RFP' button is deliberately left in English: this is the bug your LLM-as-judge translation check should find.
3. As admin, open **Data** to show the JSON files, record counts, upload size, export and reset.

7. Test hooks and API

Hook	How to use it
Deep link market and language	/projects?market=DK⟨=en (vendor users ignore ?market)
Feature flag overrides (session)	?ff_vatBreakdown=off, ?ff_aiAnalyzer=on, ?ff_vendorQuestions=on, ?ff_reset=1
Stable selectors	data-testid on every interactive and asserted element. The body has data-market, data-lang, data-locale and data-role
Config for assertions	GET /api/config returns markets, VAT, flags, scoring weights and upload limits
Translations	GET /api/i18n returns all strings and any keys a language is missing

Test data API (header x-test-token required)

Method	Path	Purpose
POST	/api/test/reset	Restore the demo data
GET	/api/stats	Data files, record counts, upload size
GET, POST, DELETE	/api/users	Create and remove users (password hashes are never returned)
GET, POST, DELETE	/api/projects	Create projects. DELETE cascades to RFPs, proposals and uploaded files
POST	/api/projects/:id/requirements	{name, priority}
GET, POST, DELETE	/api/vendors	Create and remove vendors
POST	/api/projects/:id/rfps	Create RFPs. Past deadlines are allowed, so tests can create closed RFPs
POST	/api/rfps/:id/files	Multipart upload, field name 'files'
POST	/api/rfps/:id/proposals	{vendorId, priceNet, deliveryMonths}

```
curl -X POST http://localhost:3000/api/test/reset -H "x-test-token: local-test-token"
```

8. Deploy online

Option A: Render (simplest)

1. Push the project to GitHub (commands below).
2. In Render, choose New, then Blueprint, and select the repository. Render reads `render.yaml`.

3. Enter TEST_API_TOKEN (a long random string) and DEMO_PASSWORD when prompted. ANTHROPIC_API_KEY is optional.
4. When the deploy finishes, open https://<your-service>.onrender.com/api/health. It should return status ok.
5. Add BASE_URL and TEST_API_TOKEN as GitHub Actions secrets for your test pipeline.

Free plan limits. The free instance sleeps when idle, so open /api/health a few minutes before a demo. Its disk is not persistent: data resets on every restart or deploy. For data that survives restarts, use a paid instance with a persistent disk (see the commented block in render.yaml) and set DATA_DIR to the disk mount path. Check Render's current plan details on their website.

```
git init && git add . && git commit -m "ProcureLab: vendor management demo"
git branch -M main
git remote add origin https://github.com/<you>/procurelab.git
git push -u origin main
```

Option B: Docker (any host)

```
docker build -t procurelab .
docker run -p 3000:3000 -v procurelab-data:/app/data \
  -e SESSION_SECRET=change-me -e TEST_API_TOKEN=change-me procurelab
```

The named volume keeps the JSON files and uploads across container restarts.

9. Troubleshooting

Problem	Fix
'node' is not recognized, or the version is too old	Install Node.js 22 LTS from nodejs.org and open a new terminal. Check with node -v.
Error: listen EADDRINUSE :3000	Another app uses port 3000. Start on another port: PORT=3001 npm start (Windows PowerShell: $env:PORT=3001; npm start).
Login fails after changing DEMO_PASSWORD	Passwords are hashed when the demo data is created. Run npm run reset-data, or use Data, then Reset, to apply the new password.
The app starts with old or broken data	Stop the app, delete the data folder (or run npm run reset-data), and start again.
An upload is rejected	Only PDF, DOCX, XLSX, PPTX, TXT, CSV, PNG and JPG are allowed, up to MAX_UPLOAD_MB (default 5 MB) per file and 5 files.
The test API returns 401	Send the header x-test-token with the value of TEST_API_TOKEN (local default: local-test-token).
The test API returns 503 online	TEST_API_TOKEN is not set while NODE_ENV=production. Set it in the hosting dashboard.
'Generate AI summary' shows 'not configured'	This is expected without ANTHROPIC_API_KEY. Set the key (and, if needed, ANTHROPIC_MODEL) and restart.
The online demo is slow on the first click	The free hosting instance was asleep. Open /api/health a few minutes before presenting.
Data disappeared online	The free hosting disk is not persistent. Use a persistent disk with DATA_DIR, or rely on the demo reset.

10. Known limits

- It runs as a single process with JSON files. It is not designed for many concurrent writers or multiple instances.
- There is no CSRF protection, no email delivery, and no self-service password reset. Users are managed by the admin.
- Uploads are checked by file extension and size, not scanned for malware.
- Rule-based analyzer scores are deterministic. The optional AI summary is not, on purpose, so it can be evaluated with LLM-as-judge.
- The German 'Send RFP' label is a deliberate defect for the translation-check demo. Fix it in src/i18n.js to close the loop live.

Project layout

```
server.js              app entry: loads data, sets market, language and flags
src/config.js          environment settings
src/db.js              JSON file persistence (atomic writes, one file per collection)
src/store.js           domain logic: users, vendors, projects, RFPs, proposals, files
src/seed.js            fictional demo data and generated demo PDFs
src/insights.js        proposal analyzer: scores, risk, missing info, AI summary
src/uploads.js         upload rules (types, size, count)
src/i18n.js            EN, NL, DA, DE translations
src/routes/            ui.js (pages), admin.js (users, data), api.js (test data API)
src/views/             server-rendered HTML
public/                styles.css, app.js
scripts/reset-data.js  npm run reset-data
docs/                  this guide
```